Instituto Tecnológico de Costa Rica
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación
Curso: IC-6600 Principios de Sistemas Operativos
Profesor: M.Sc. Esteban Arias Méndez

Proyecto Programado

Entregable 2: Prototipo 1 y ajustes de arquitectura

Implementación de un Sistema de Memoria Compartida Distribuida (DSM) con Paginación bajo Demanda y Coherencia MSI

Estudiantes:
Sergio Andrés Zúñiga Castillo, carné 2024146884
José Ignacio Madriz Ortiz, carné 2024089190

Periodo: II Semestre 2026
Fecha de entrega: Noviembre 13, 2026

Abstract
This report documents the first prototype and configuration, node startup, the node table, the three TCP listeners, and periodic heartbeat exchange. A demonstration message is used to test transport over the data and control channels. Early API implementation and MSI policy scaffolding are distinguished from pending DSM functionality. This prototype's purpose is to establish a working messaging protocol, proper security checks and TCP communication. The core architecture remains unchanged; implementation and configuration adjustments are documented separately from an experimental WSL/NAT variant.

Índice

2.1 Nodo y canales....3
2.2 Hilos y responsabilidades....3
3.1 Archivo de configuración....4
3.2 Tipos de sección 4....5
4.1 Encabezado y framing....6
4.2 Heartbeat....6
4.3 Mensajería de demostración....6
9.1 Compilación e inicio....10
9.2 Pruebas ejecutadas....10

1. Introducción y alcance

El Entregable 1 definió un DSM en espacio de usuario para Linux: cada ejecución es un nodo par, los hosts comparten una configuración, las páginas tienen un nodo home determinista y la coherencia se plantea con MSI. El objetivo de este entregable es la primera implementación ejecutable de esa arquitectura y una descripción fiel de su estado actual, no afirmar que el DSM completo esté terminado.

El prototipo se encuentra en `prototipo.cpp` y expone el borrador C de la API, que de momento no logra nada, mediante `dsm.h`. Cada proceso se inicia con un archivo de configuración y un identificador de nodo. El alcance funcional demostrado es el arranque, la detección de pares por heartbeat y el intercambio de mensajes de prueba por TCP. No se transfieren páginas ni se ejecutan asignaciones DSM reales, pero la estructura para mandar y recibir las peticiones ya está en pie y probada.

La implementación usa C++17, sockets POSIX/Linux, `poll` y `std::thread`. Por ello, esta versión no es un ejecutable nativo de Windows. Para prubeas fuera del laboratorio, WSL ofrece un entorno Linux, pero su NAT requiere configuración de red adicional que se documenta en la sección 7.

2. Arquitectura implementada

2.1 Nodo y canales

Un nodo continúa siendo una instancia del mismo programa. Su identificador es la posición de su host en la configuración compartida. En el arranque, la clase `Node` valida el identificador, resuelve los hosts a IPv4, prepara la tabla local y abre tres sockets TCP de escucha. Para el nodo *k*, los puertos se calculan como `port_base + 3*k + rol`, con rol 0 para datos, 1 para control y 2 para heartbeat.

Rol	Puerto para node_id = k	Uso implementado
Datos (0)	port_base + 3*k	Mensajes de demostración por TCP.
Control (1)	port_base + 3*k + 1	Mensajes de demostración por TCP.
Heartbeat (2)	port_base + 3*k + 2	Detección periódica de disponibilidad.

Con el `port_base` asignado de 28400, los nodos 0 y 1 escuchan, respectivamente, en 28400–28402 y 28403–28405. Para 32 nodos, el último puerto utilizado es 28495, dentro del rango asignado 28400–28499.

2.2 Hilos y responsabilidades

Hilo/componente	Estado en el prototipo
Hilo principal	Inicia el nodo y atiende la consola: nodes, broadcast, send y quit.
Hilo de red	Implementado. Un único listener usa poll para los tres sockets y las conexiones aceptadas; lee encabezados y cargas sin bloquear esperando el frame completo.
Hilo de heartbeat	Implementado. Envía un heartbeat a cada par configurado cada heartbeat_ms y revisa vencimientos.
Hilos de usuario	La clase conserva una operación para lanzar una función de aplicación como hilo, pero la consola todavía no implementa el comando run.
Hilos de servicio	Se mantienen los tipos, el arreglo de locks por página y el esqueleto de servicio, todavía no se despachan solicitudes DSM ni se esperan respuestas de invalidación.

El heartbeat actual es bidireccional: cada proceso escucha en su puerto y también abre conexiones salientes hacia los demás. Cuando el listener recibe tráfico de un peer válido por el canal correcto, actualiza la marca de actividad. Si el silencio supera timeout_ms, el estado se muestra como *CAÍDO y* un mensaje posterior lo vuelve a marcar *EN LÍNEA*.

3. Configuración, estructuras y datos de nodo

3.1 Archivo de configuración

El analizador lee líneas clave = valor, permite comentarios que comienzan con # y admite varias líneas host. El orden de esas líneas fija los node_id y debe ser idéntico en todos los nodos. Se valida el rango de hosts, tamaños positivos, alineamiento de page_size, capacidad de páginas y desbordamiento de puertos.

Campo	Valor por defecto/estado	Descripción
dsm_base	0x100000000000	Dirección base prevista para la memoria DSM, aún no se reserva ni se mapea.
page_size	4096 bytes	Tamaño de página múltiplo de 4096.

pages_per_node	256	Capacidad de páginas asignada a cada home.
heartbeat_ms / timeout_ms	1000 / 5000 ms	Período de envío y de caída.
port_base	28400	Primer puerto del bloque de tres por nodo.
host	1 a 32 entradas	IPv4, su posición en la lista es el identificador del nodo.

Todos los nodos deben compartir la misma lista de hosts en el mismo orden. Para procesos en una sola máquina se puede repetir 127.0.0.1.

3.2 Tipos de datos abstractos

El código declara las estructuras de tamaño fijo explicados en la sección 4 y verifica sus tamaños con static_assert. NodeInfo se mantiene como entrada de la tabla de nodos. Se implementa, además, un arreglo local de PageEntry inicializado como Invalid, todos cimientos para la implementación MSI.

Tipo	Tamaño validado	Uso actual
MsgHeader	16 B	Reserva el formato del protocolo DSM.
NodeInfo	16 B	Host, rango de páginas y estado de cada nodo.
PageEntry	16 B	Tabla local inicializada en I. Inactivo.
NodeStats	32 B	Contadores iniciales de capacidad/estado. Inactivo.
AllocReq	8 B	Declarado para futuras solicitudes de asignación. Inactivo.
QueryReq	12 B	Declarado para futuras consultas. Inactivo.
Pending	64 B en Linux x86-64/glibc	Respuestas asíncronas futuras. Inactivo
Config	2080 B	Representación fija de configuración validada por tamaño.

La tabla de nodos protege los datos y marcas de último contacto con un mutex. Inicializa para cada entrada el node_id, el rango first_page/num_pages y su estado. Las estadísticas iniciales son una referencia de configuración, no métricas de transferencia ya realizadas.

4. Transporte TCP y protocolo del prototipo

4.1 Encabezado y framing

Los frames comienzan con el encabezado binario de 16 bytes descrito en la sección 5: type, sender, status, flags, seq, page y payload_len. Los campos de 32 bits se convierten a orden de red. Como TCP es un flujo de bytes, el receptor acumula exactamente el encabezado y después la cantidad indicada por la carga. Los frames incompletos permanecen asociados a la conexión y se retoman en otra iteración de poll.

Las solicitudes estándar de las tablas de la sección 5 (JOIN, PAGE_REQUEST, INVALIDATE, ALLOC_REQUEST, FREE y QUERY_REQUEST, entre otras) están definidas como tipos, pero todavía no tienen handlers ni cargas implementadas. El protocolo real de operaciones DSM sigue pendiente.

4.2 Heartbeat

El heartbeat usa el tipo 3, no lleva carga, usa PAGE_NONE y circula únicamente por el puerto heartbeat. Se emite una conexión TCP por par y por período, el listener acepta, valida el frame y actualiza la tabla de nodos. Los tiempos de espera y cierres se procesan sin bloquear el hilo que atiende los otros sockets.

4.3 Mensajería de demostración

Para comprobar el transporte, se añadió el tipo experimental 0x80, fuera de los tipos DSM 1–12. Acepta texto de hasta 1024 bytes (y no mayor que page_size) por los canales de datos o control. Un broadcast se implementa como un envío unicast separado a cada peer. El receptor imprime el ID del emisor, canal, modalidad y texto en la terminal.

Comando de consola	Comportamiento
broadcast data <mensaje>	Envía el texto por el canal de datos a todos los demás nodos.
broadcast control <mensaje>	Envía el texto por el canal de control a todos los demás nodos.
send <node_id> data <mensaje>	Envía el texto por datos a un nodo.
send <node_id> control <mensaje>	Envía el texto por control a un nodo.

Esta mensajería verifica establecimiento TCP, framing, carga útil y despacho por canal y no confirma recepción al emisor. El mensaje de “enviado” confirma que el frame se pudo escribir en la conexión, no que la aplicación destino lo haya procesado.

5. Políticas de coherencia MSI

El prototipo incluye funciones que expresan qué transición y coordinación corresponderían a un acceso, pero no instala páginas, no invalida copias por la red y no modifica la tabla según solicitudes reales.

6. API: estado de implementación

dsm.h expone los tipos C compatibles y las firmas del borrador de la sección 6. El estado funcional es parcial:

Función	Estado del prototipo
dsm_init, dsm_shutdown	Implementan lectura de configuración, creación/inicio y cierre del nodo.
dsm_nodes	Devuelve una copia local de la tabla configurada y sus estados de heartbeat.
dsm_stats	Devuelve los valores iniciales locales. Sin implementar
dsm_page_table	Permite leer el rango local inicializado. Sin implementar
dsm_page_size, dsm_strerror	Implementadas para tamaño configurado y descripción de códigos.
dsm_alloc, dsm_free	Validan argumentos básicos, pero devuelven DSM_E_NOT_IMPLEMENTED hasta contar con memoria DSM y protocolo de asignación/liberación.

El código DSM_E_NOT_IMPLEMENTED evita que las operaciones incompletas aparenten haber tenido éxito. La API aún no implementa mmap/memfd, reserva del rango compartido, manejo de SIGSEGV, semáforos de solicitudes pendientes ni comunicación de consultas remotas.

7. Variante experimental para WSL

La variante de la rama WSL-testing se mantiene separada de la arquitectura base. WSL2 en modo NAT presenta una IP privada interna que no es directamente alcanzable desde la red LAN. Para recibir conexiones desde otro equipo, se requiere exponer la IP LAN de Windows y reenviar cada puerto del nodo hacia la IP actual de WSL, además de habilitar los puertos entrantes en Windows Firewall.

Para el nodo 1, con port_base = 28400, el reenvío requerido es:

Servicio	Puerto Windows/LAN → WSL
Datos	28403 → 28403
Control	28404 → 28404
Heartbeat	28405 → 28405

La IP interna de WSL puede cambiar después de reiniciar WSL o Windows, por lo que se debe revisar y actualizar el destino del portproxy. La dirección que se configura como host del nodo remoto es la IP LAN de Windows, que es el punto alcanzable por el otro equipo.

El NAT observado reescribe el origen de conexiones reenviadas: WSL recibió tráfico del host virtual/gateway (por ejemplo, 172.23.0.1) en lugar de la IP LAN del emisor. La validación estricta por coincidencia entre IP de origen y host configurado funciona en la LAN directa, pero rechaza ese tráfico reenviado. La variante experimental evita la comparación de IP y acepta el ID sender declarado en el encabezado, si corresponde a un nodo configurado y no es el propio nodo.

Compromiso de seguridad: el encabezado no tiene autenticación. Un tercero que alcance el puerto puede falsificar el campo sender, por ello, esta rama de prueba reduce la verificación de origen y no es apropiada para exponer el prototipo a Internet. El port forwarding resuelve direccionamiento solamente. Esta excepción es únicamente una adaptación para usar WSL y no modifica la arquitectura base.

8. Cambios y ajustes respecto al diseño original

La arquitectura DSM general se conserva: un nodo par por ejecución, tabla distribuida de nodos, home fijo por rango, tres roles de puerto, heartbeat y separación entre listener y trabajo que potencialmente espera. Como ajuste menor de arquitectura, se añadió una validación de identidad al inicio: el node_id seleccionado debe corresponder a una IPv4 configurada en la máquina local antes de abrir los puertos. Esto evita que un proceso arranque haciéndose pasar accidentalmente por otro nodo.

Los ajustes de esta primera implementación son:

1. **Rango de puertos asignado:** el ejemplo inicial empleaba un puerto base provisional; se actualizó el valor por defecto a 28400 conforme al rango 28400–28499 asignado. Se conserva la fórmula de tres puertos por nodo.
2. **Frame de demostración aislado:** el tipo experimental 0x80 se agregó únicamente para validar envío dirigido y difusión lógica por datos/control. No sustituye ni altera

los tipos DSM.

3. **Inicio desde consola:** se reciben ruta de configuración e ID de nodo como argumentos.
4. **API parcial y error explícito:** se implementan las operaciones de ciclo de vida y consultas locales posibles en esta etapa.
5. **Variante WSL aislada:** ajuste de validación por ID, se documenta aparte y no se considera un cambio al diseño, existe como una rama separada.
6. **Validación de identidad local:** antes de abrir puertos, el arranque comprueba que la IPv4 asociada al node_id elegido esté asignada a la máquina. Si no coincide, el nodo falla con un mensaje claro en lugar de iniciar con una identidad que pertenece a otro host.

9. Trabajo pendiente

El trabajo necesario para completar el DSM sigue siendo sustancial: implementar JOIN/JOIN_ACK, operaciones reales del protocolo, serialización por campo, asignación contigua en el home, directorio de copias MSI, envío de páginas e invalidaciones con ACK, tabla de pendientes, memoria compartida con memfd/mmap, protecciones mprotect, manejador de fallos y sincronización, API de asignación/liberación y consultas remotas, junto con pruebas de concurrencia y fallos.